

SOFTWARE ARCHITECTURE

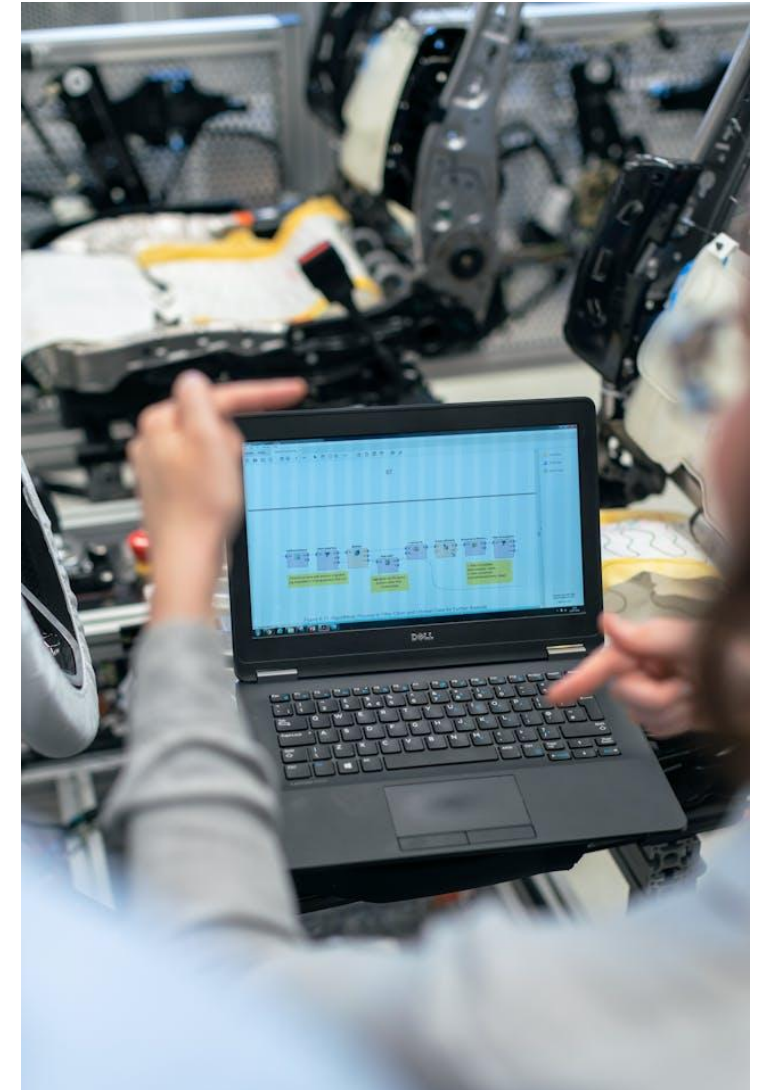
SYSTEM INTEGRATION

Arturo Mora-Rioja
amri@ek.dk

EK

Software Architecture

- The choice of a system design pattern affects the whole system
 - Maintainability
 - Deployability
 - Scalability
 - Testability
- Historically tied to hardware and networking architecture



Software Architecture Models

- Monolith
- N-Tier
- Enterprise SOA
- Microservices
- Serverless
- P2P



Monolith

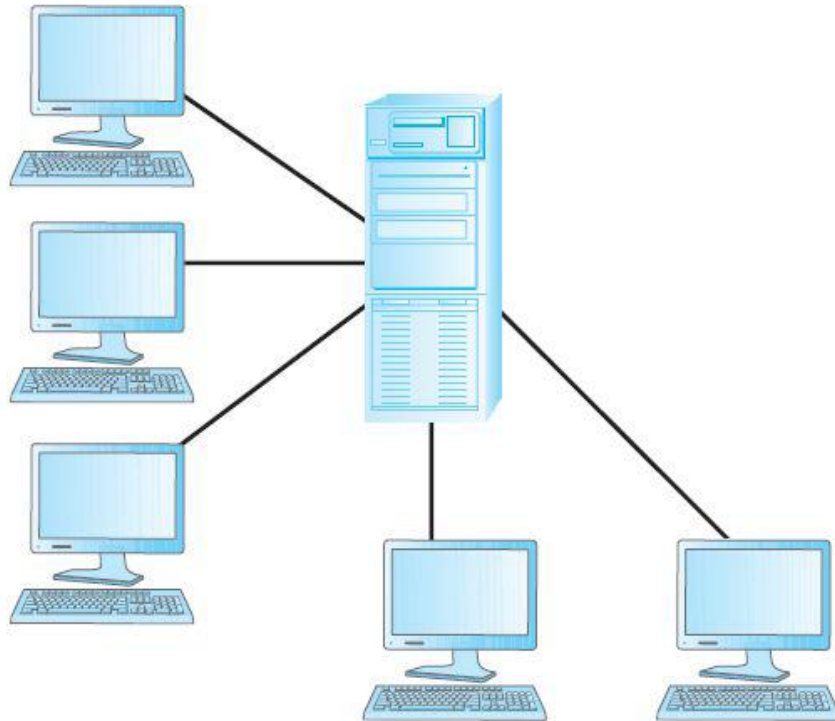
- Single executable
- Projects with limited scope and internal web applications
- 👍
 - Easy to understand
 - Easy to implement
 - Easy to deploy
 - No need for coordination with different teams
- 👎
 - Tight coupling
 - Often results in complex, difficult to maintain code
 - One-size-fits-all approach
 - Usually tied to one specific programming language



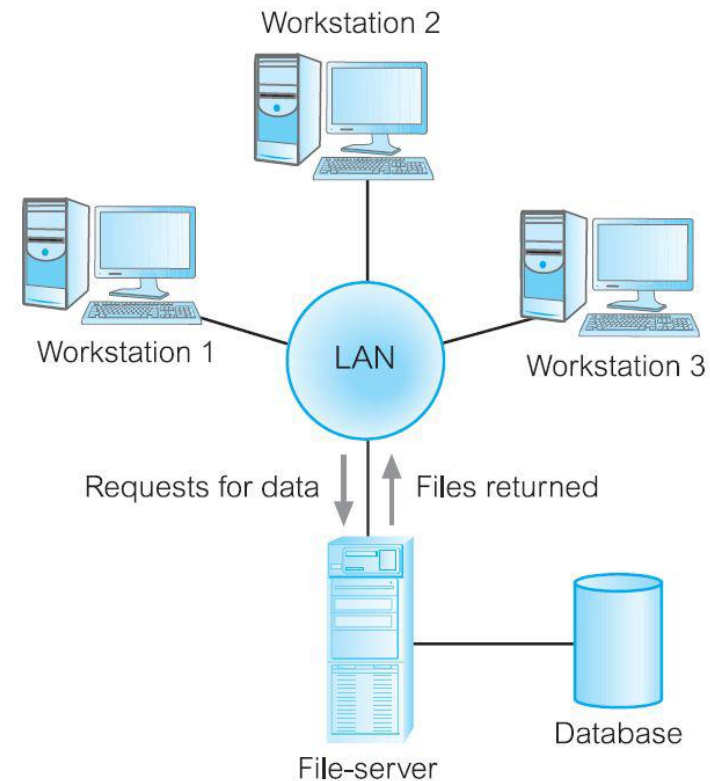
© [Patrick Boyer](#) @ Pexels

Traditional Infrastructure Architectures

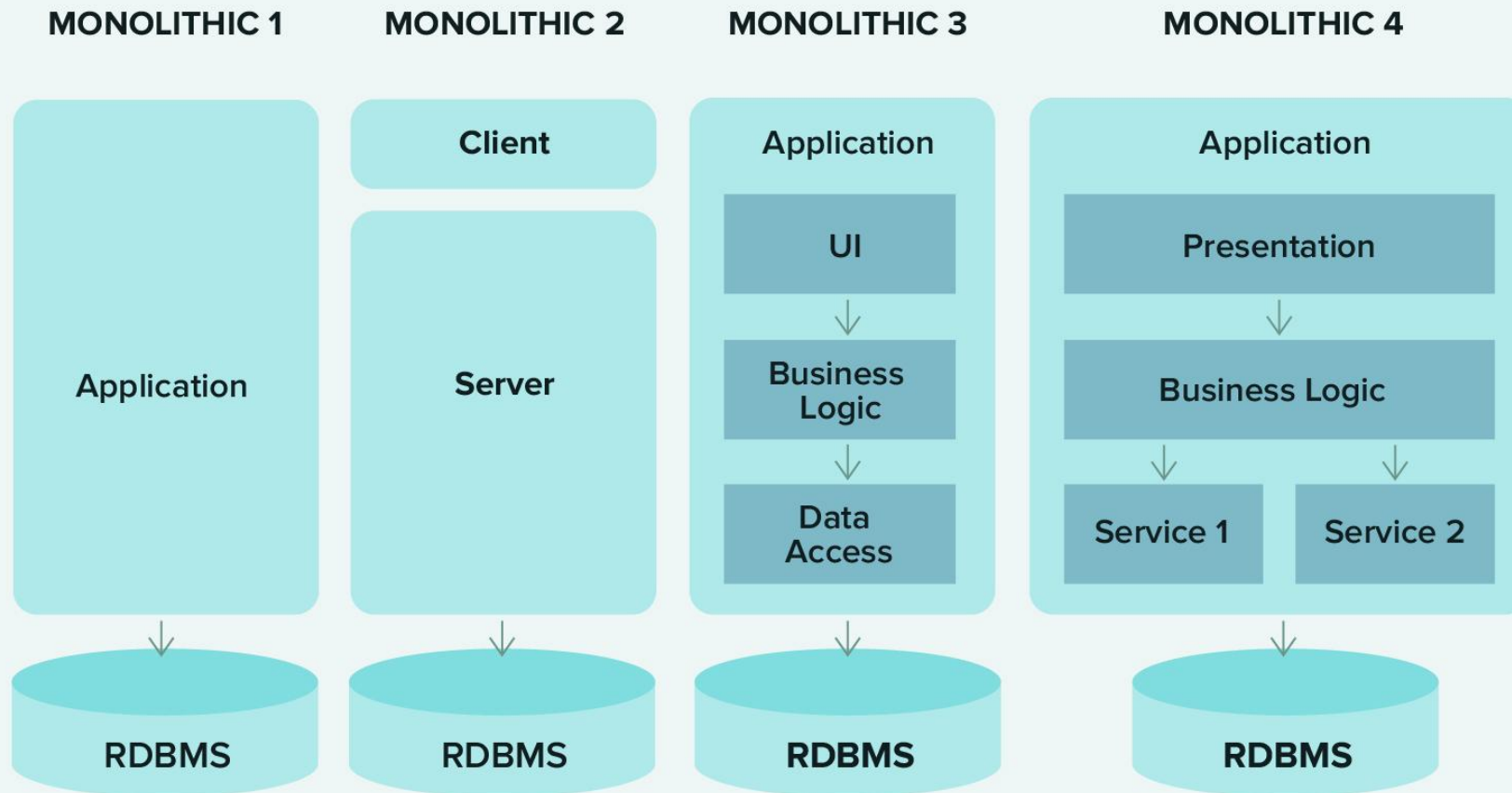
- Teleprocessing (approx. 1965-1985)
 - CPU-less “dumb terminals” connected to a central computer
 - Centralised model



- Client-Server (approx. 1985-2000)
 - Workstation connected via a LAN
 - Applications installed locally



Monolith – Possible Internal Structures

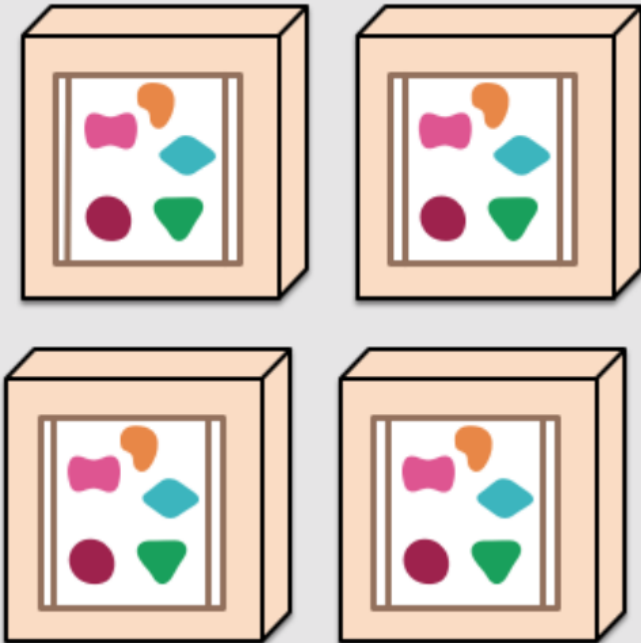


Monolith Scaling

A monolithic application puts all its functionality into a single process...



... and scales by replicating the monolith on multiple servers



- Database scaling options and challenges
 - Vertical scaling
 - One single database in one server
 - Expensive to maintain
 - Single point of failure
 - Downtime expected
 - Horizontal scaling
 - Replication
 - Real-time consistency issues
 - Sharding
 - Complex cross-server SQL queries

© 2014 Fowler

Monolith Examples

- Shopify (core application)
- Stack Overflow
- CMSs
 - WordPress
 - Drupal
 - Joomla
 - Moodle
- Wikipedia
- Magento / Adobe Commerce



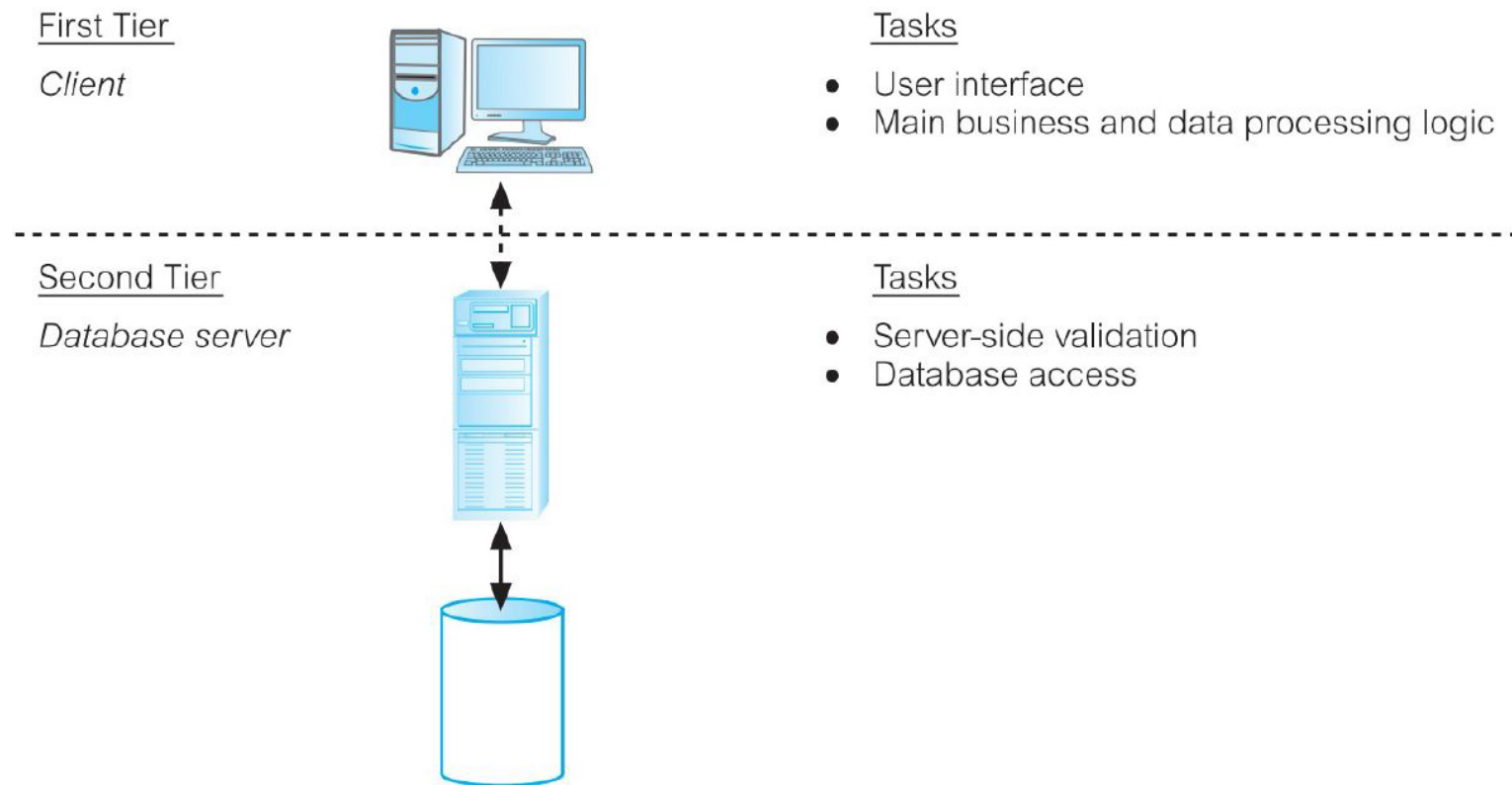
© YES @ Pexels

N-Tier = Multi-Tier

- Divided into multiple tiers, each one performing different system functions (technical, not business-oriented division)
- It allows for physical separation of tiers
- 3-tier (Presentation – Business – Data). Popular in web development (approx. 1990-2005)
- 👍
 - Independent development for each tier
 - Independent integration, deployment, and scalability for each tier
- 👎
 - Changes in one tier may affect the others
 - E.g., a new form control may affect the business logic and the corresponding value may need database handling

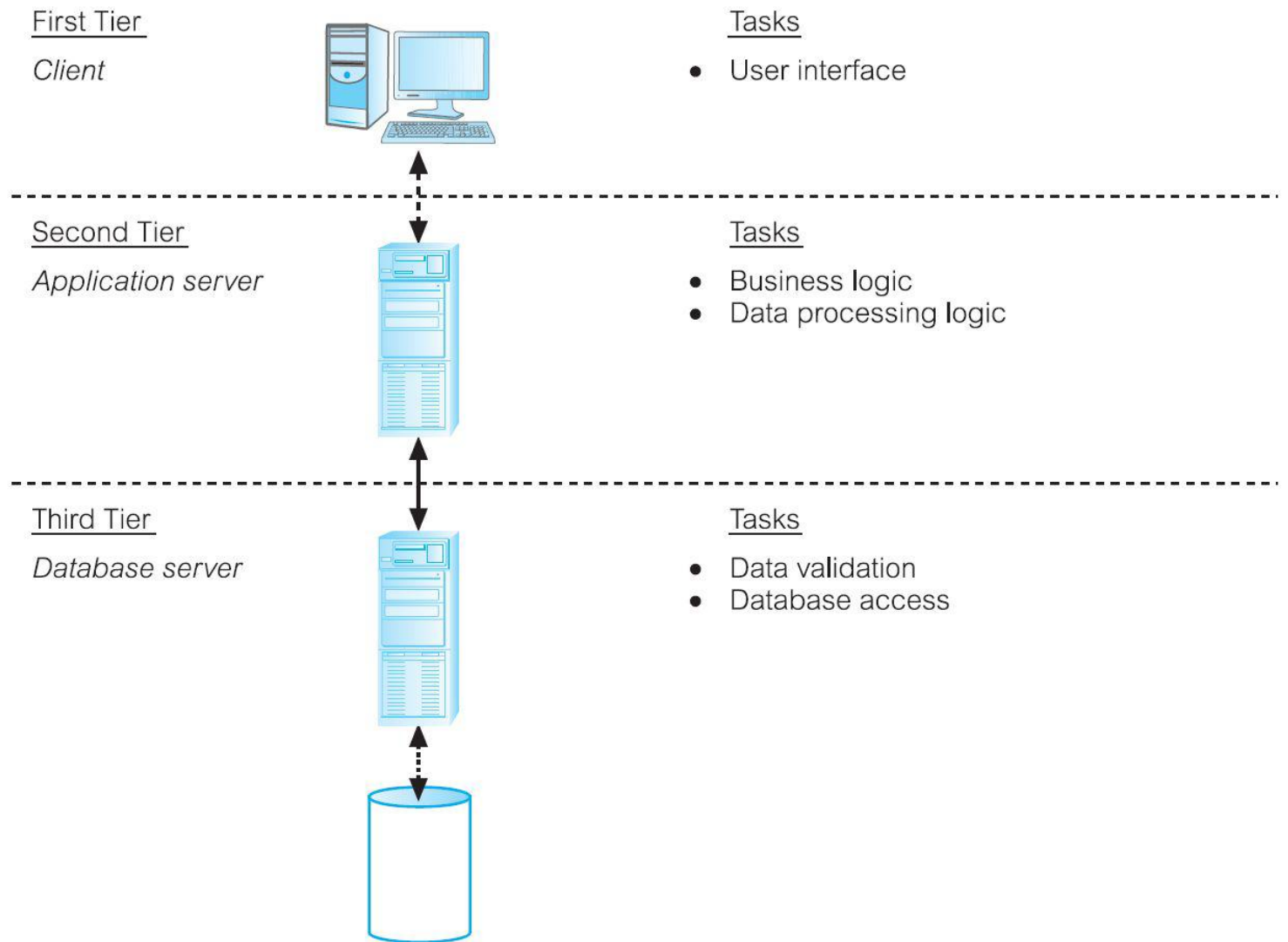
N-Tier Infrastructure Architectures

- 2-Tier Client-Server
 - Only data requests are exchanged with the server



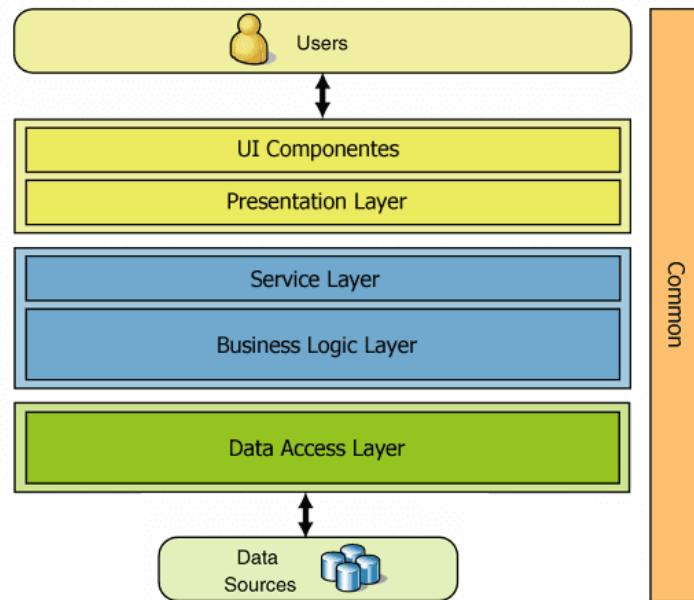
N-Tier Infrastructure Architectures

- 3-Tier Client-Server
 - Thin client
 - Natural adaptation to the web environment
 - Popularised in the 1990s

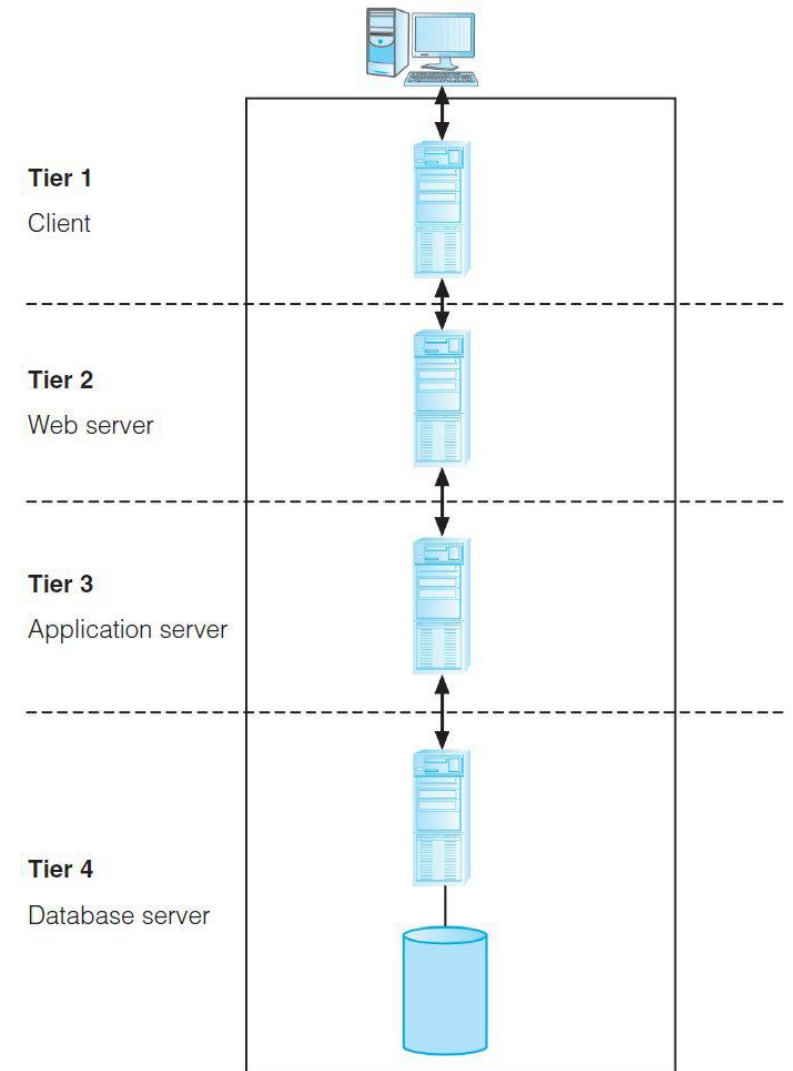


N-Tier Infrastructure Architectures

- N-Tier Client-Server
 - Flexible
 - Neither the number of tiers or their functions are predefined
 - Scalable



© 2010 Novikov



© 2015 Connolly and Begg

EK

10-Tier Example

Client	Web browser, mobile app, external system
Presentation	View rendering (responsive UI, mobile UI)
Security	Authentication, input validation and formatting
Controller	API gateway, REST controllers, GraphQL resolvers, gRPC endpoints
Application	Business services (e.g., OrderService, UserAccountService, PaymentService)
Domain model	Business logic, independent from infrastructure
Audit	Logging
Cache	Either in-memory or distributed (e.g., Redis)
Persistence	ORMs, data mappers across different database models, queries
Database	DBMSs, stored procedures, triggers, views, SQL DDL

N-Tier Examples

- 3-tier
 - SAP S/4HANA
 - Oracle E-Business Suite
 - Microsoft Dynamics 365 Business Central (on-premise)
 - Oracle Forms Services
- 4-tier
 - PeopleSoft
- 5-tier
 - IFS Planning and Scheduling Optimization (IFS PSO)
- 6-tier
 - SAP BusinessObjects Business Intelligence Platform 2025



Enterprise SOA (Service-Oriented Architecture)

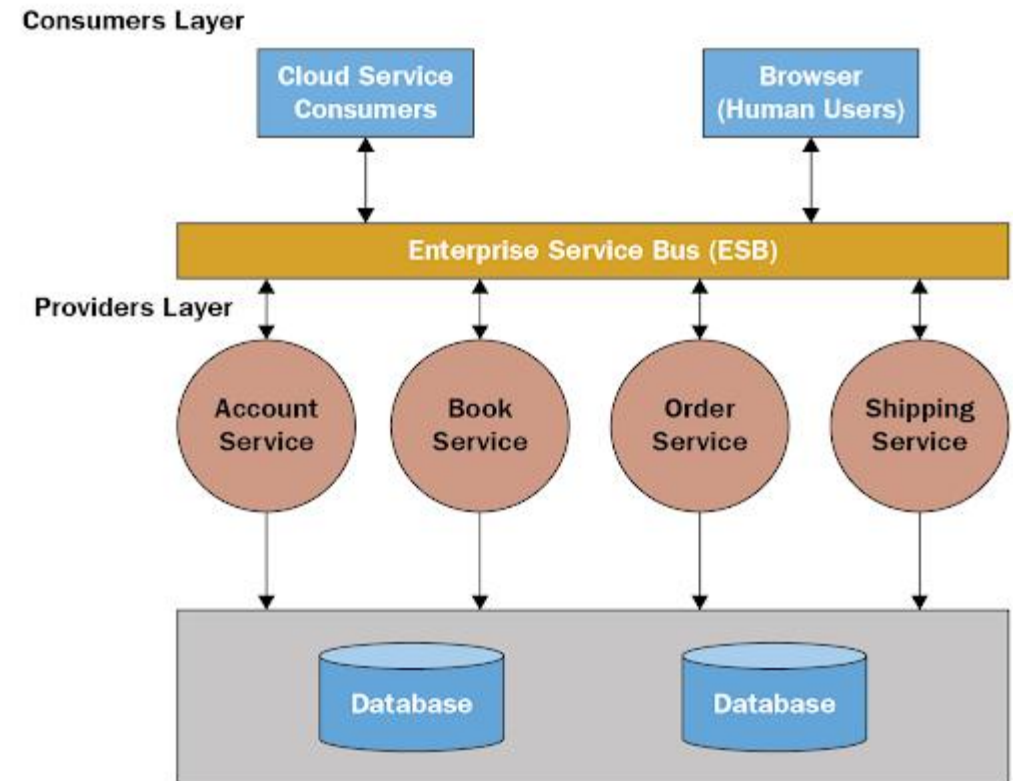
- Multiple services, each one representing a specific business activity (approx. 2000-2015)
- Ability to create composite services from individual services
- Contract standardisation and central governance



- Loose coupling
- Easy to scale
- No duplication of functionality



- Central governance implies reduced agility and team autonomy
- Costly to implement (establishing the ESB – Enterprise Service Bus and the contracts)



© [O'Reilly](#)

Enterprise SOA Examples

- Oracle SOA Suite 14c
- Microsoft BizTalk Server 2020
- IBM App Connect Enterprise 13
- TIBCO ActiveMatrix BusinessWorks 6.12
- SAP Process Orchestration 7.5

ORACLE
SOA Suite



TIBCO
ActiveMatrix

SAP PO

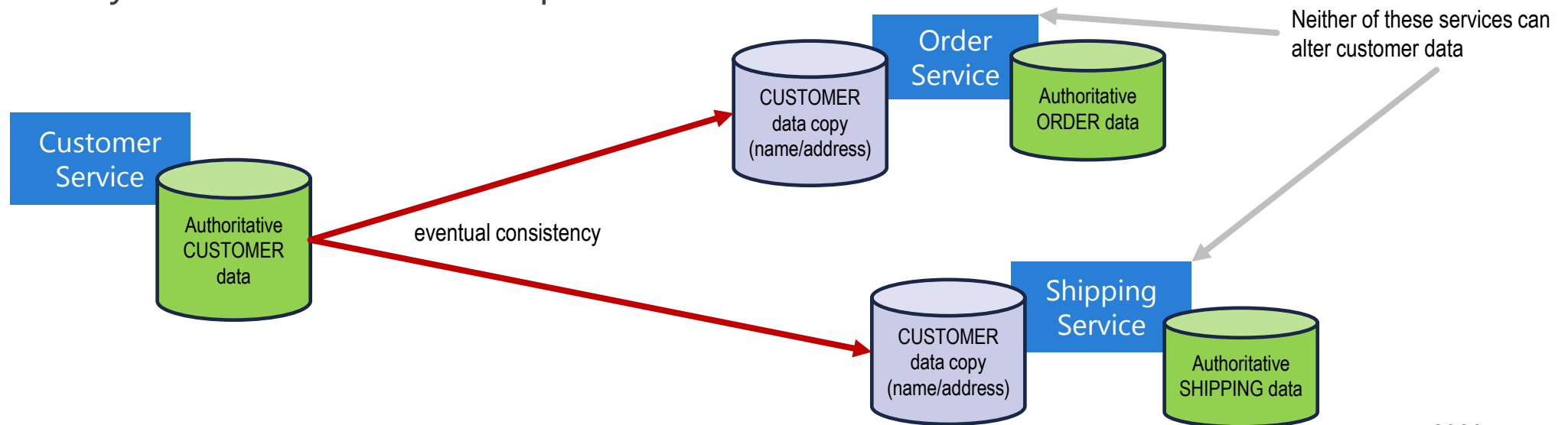


© [Vova Kras](#) @ Pexels

EK

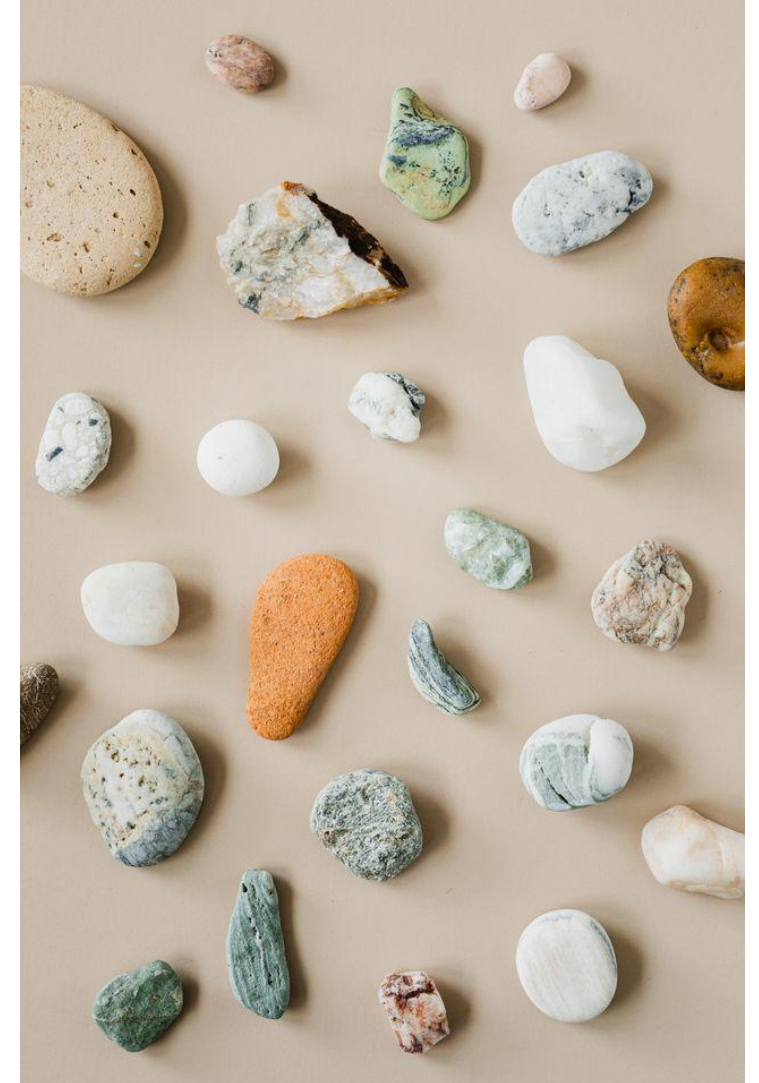
Microservices

- Evolution of Enterprise SOA (approx. 2010-present)
- No logic-heavy ESB is necessary. HTTP or a message broker is enough
- Requires automation because of complexity
- Each microservice must own its data
 - However, to keep each service totally independent from others, data may need some level of duplication



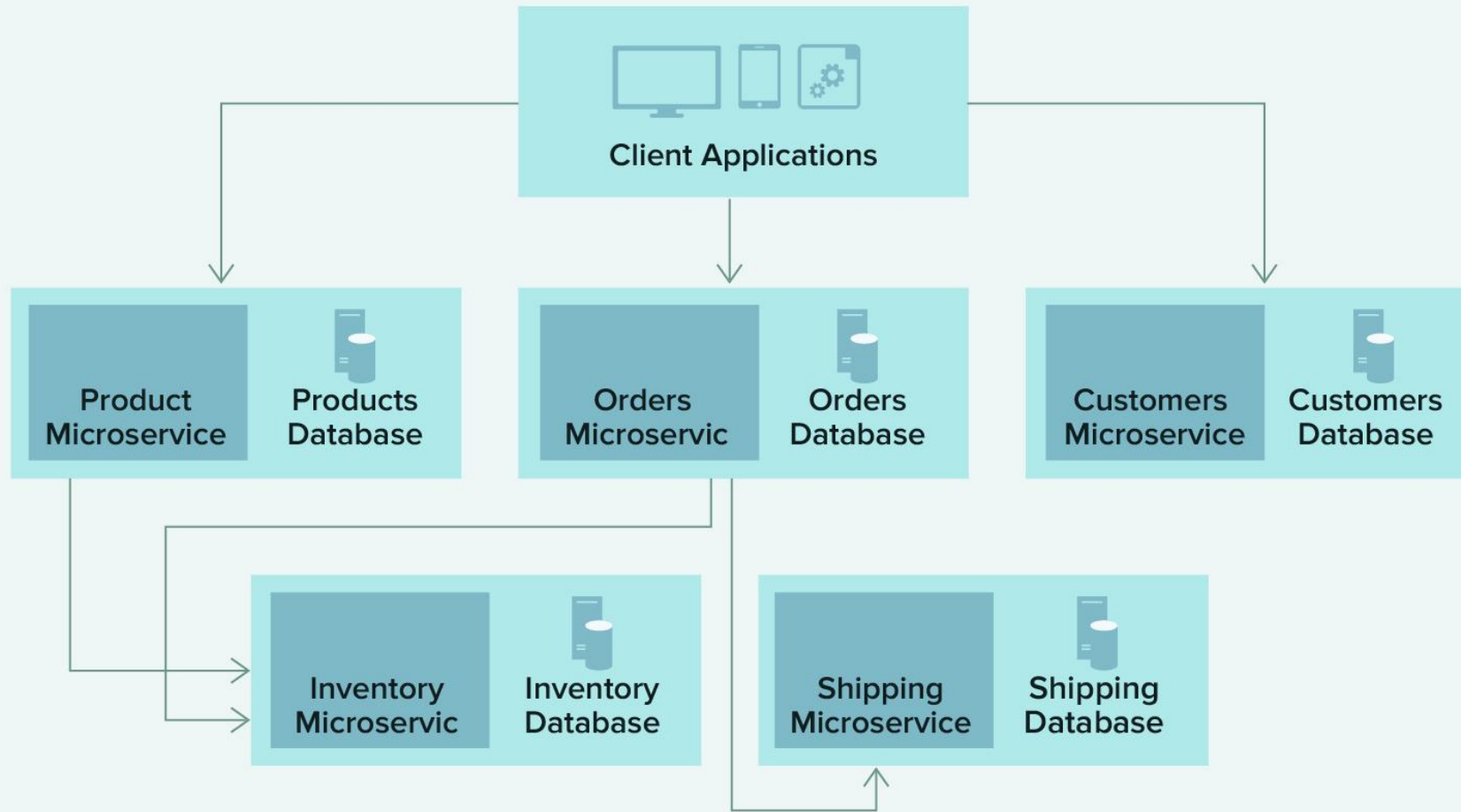
Microservices

- 👍
 - Loose coupling
 - Easy to scale
 - Increased agility (no central service)
 - Reliability due to automation (minimisation of human error)
- 👎
 - Complexity in design and implementation
 - Complexity in communication among services



© <https://kaboompics.com> @ Pexels

Microservices Example

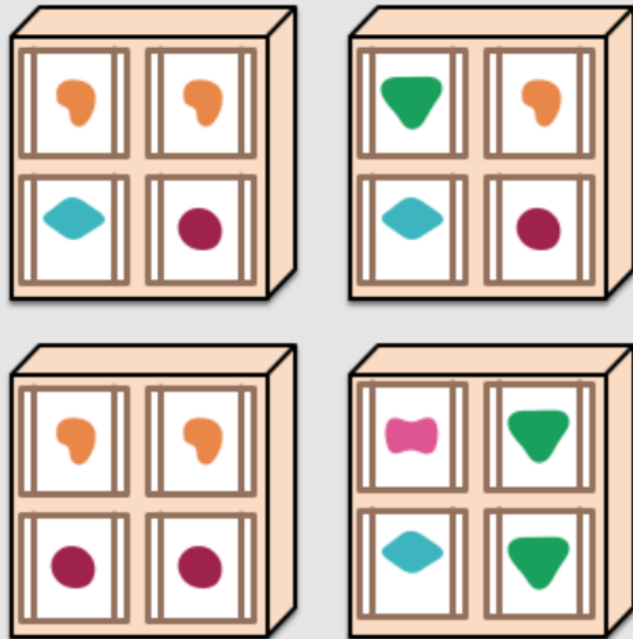


Microservices Scaling

A microservices architecture puts each element of functionality into a separate service...



... and scales by distributing these services across servers, replicating as needed.



- Database scaling options and challenges
 - Individual microservice databases may encounter the challenges of vertical scaling
 - Since each microservice has its own database, there is some level of data duplication, thus facing the challenges of horizontal scaling

© 2014 Fowler

Microservices Examples

- Netflix
- Spotify
- DoorDash
- Mercari
- Zilliz Cloud
- Nomadis
- Namshi
- Dolphin Technologies
- radicant
- CrossTech
- Pizza Hut digital platform

NETFLIX

DOORDASH

zilliz

نمشي
NAMSHI

radicant

Pizza
Hut

Spotify

mercari

NOMADIS

dolphin

CrossTech



© [elif s.](#) @ Pexels

EK

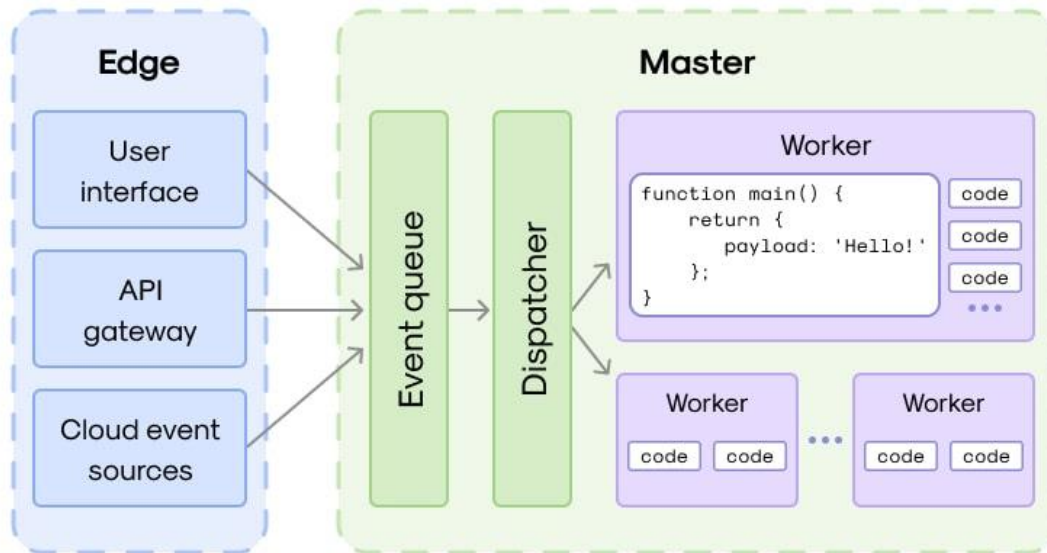
Serverless

- Cloud services are outsourced (approx. 2014-present)
 - Automatic scaling without server provisioning. Pay-per-use
 - Indicated for applications with unpredictable traffic
- Implementations
 - FaaS (Function as a Service). Aprox. 60-65% market share
 - Multiple functions, each one in a container managed by a cloud platform
 - The functions can interact with other cloud-based third-party services
 - BaaS (Back End as a Service). Aprox. 35-40% market share
 - The application implements business logic but uses third-party services for other tasks (authentication, storage)
 - Others (serverless databases, object storage, event services, workflows, serverless containers, API gateways, queues)

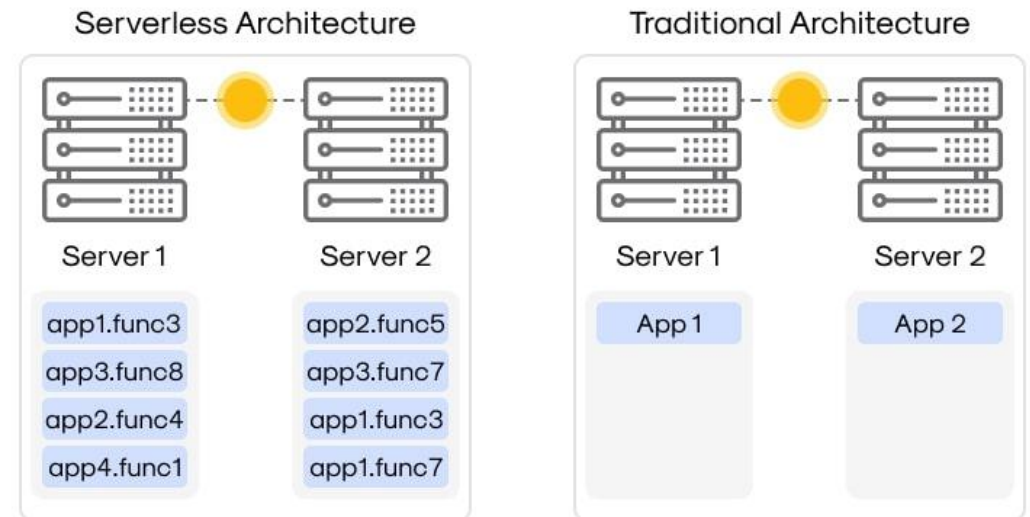
Serverless



FaaS

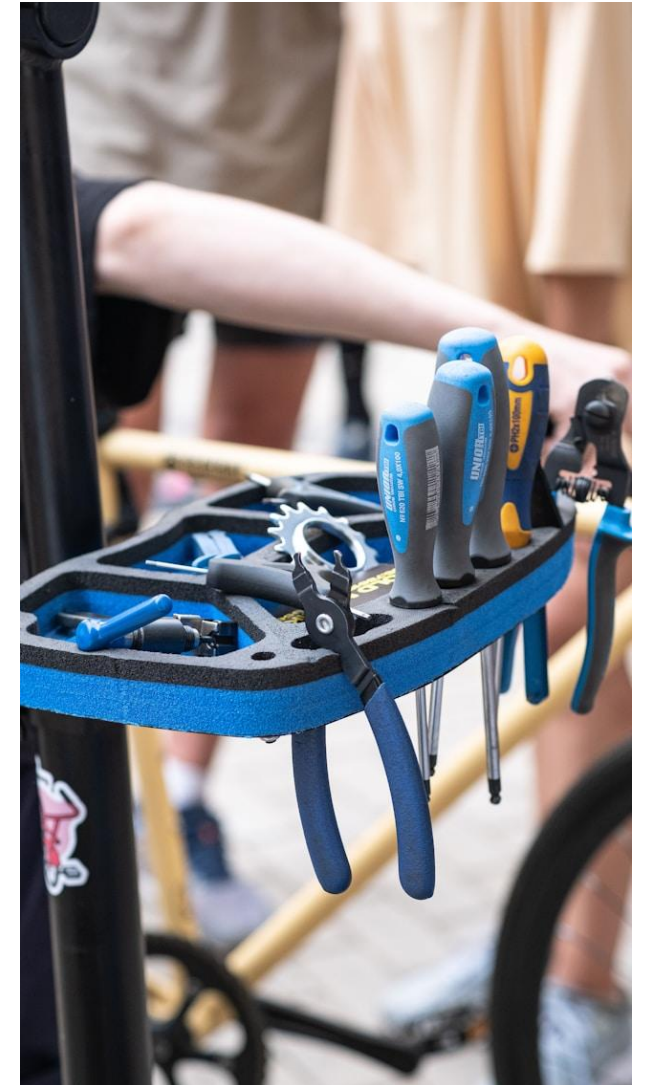


Serverless vs Traditional Architecture



Serverless

- 👍
 - Easy to scale
 - Easy to add/try new functions
 - Less or no infrastructure to administer
- 👎
 - Tied to vendor specifications
 - Steep learning curve
 - Cold Starts
 - The first time a function runs it takes longer, as it may fire up connecting services
 - Could be expensive, depending on providers



© [Anton Savinov](#) @ Unsplash

Serverless

- Design recommendations for serverless applications
 - Event-Driven Architecture
 - Statelessness
 - Small, single-purpose functions
- Main providers
 - AWS Lambda
 - Azure Functions
 - Google Cloud Functions
 - Oracle Cloud Functions



AWS Lambda



Azure Functions



Google Cloud Functions



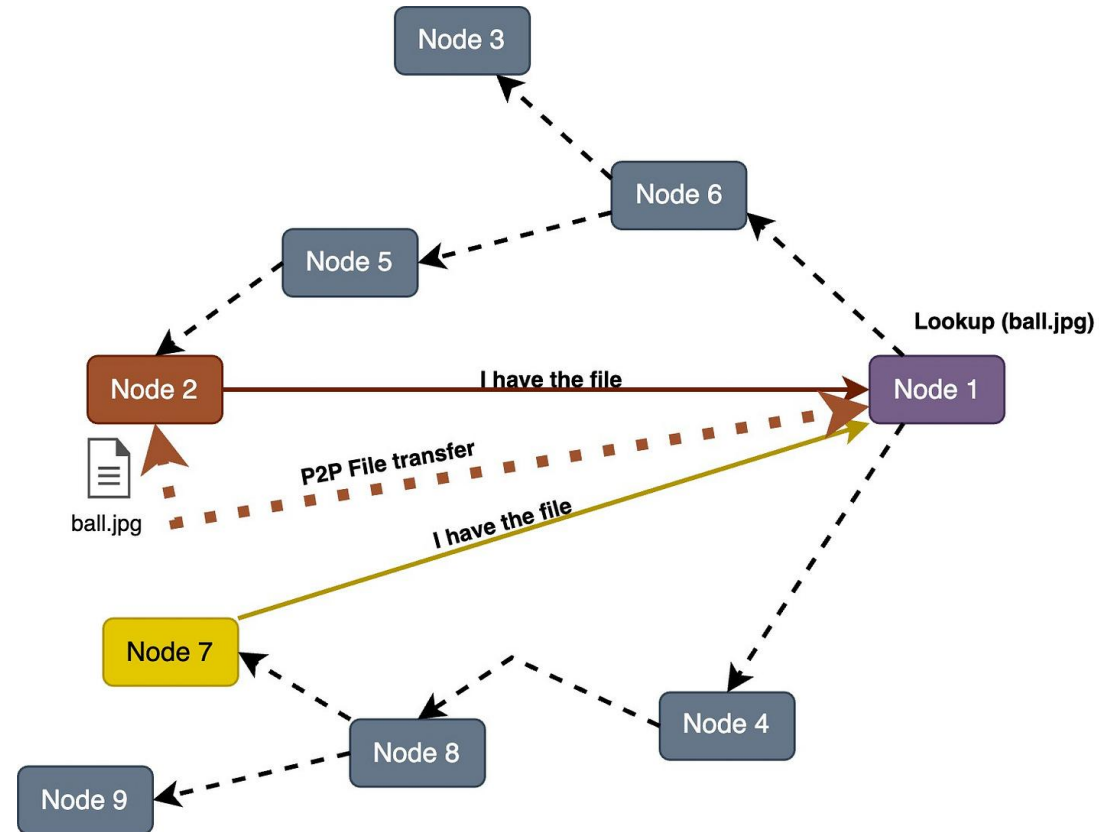
Serverless Examples

- Capital One check-clearing application
- Coca-Cola Freestyle Mobile Pour
- Guru fintech application
- iRobot connected-device backend
- FINRA market-data validation system
- Toyota Connected Mobility Services Platform
- Thomson Reuters usage-analytics service
- Neiman Marcus serverless applications



Peer-to-Peer

- No central server (approx. 1999-2010)
 - A client-server approach cannot be used
- No continuous communication
- Dynamic discovery
- 👍
 - Ideal for sharing resources (e.g., files, memory, CPU)
 - Cost-effective. No central server to maintain
 - Easy to scale
- 👎
 - Lack of central control
 - Potential security issues



Peer-to-Peer Examples

- BitTorrent
- BitTorrent File System (BTFS)
- WebTorrent
- Bitcoin
- Ethereum
- IPFS
- Syncthing
- Resilio Sync



© [Sincerely Media](#) @ Unsplash

Sources

- **Beschokov, Mukhadin.** ["What Is Serverless Architecture?"](#) Wallarm, 2025.
- **ByteByteGo.** ["A Crash Course in P2P"](#). ByteByteGo Newsletter, 2024.
- **Connolly, Thomas, and Carolyn Begg.** *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed. Pearson, 2015. ISBN: 978-1-292-06118-4.
- **Geeks for Geeks.** ["Serverless Architecture"](#). GeeksforGeeks, 2025.
- **Lewis, James.** ["Microservices"](#). MartinFowler.com, 2014.
- **Morlion, Peter.** ["Software Architecture: Patterns for Developers"](#). LinkedIn Learning, 2025.
- **Novikov, Daniil.** ["Open Source Extensible Enterprise n-tier App Framework with Multilayered Architecture"](#). Code Project, 2010.
- **Svitla Team.** ["Monolith vs Microservices Architecture: Which One to Choose?"](#) Svitla, 2024.
- Cover picture by [Expect Best](#) from [Pexels](#)